When Should a Router Commit? Binding Time in Disaggregated LLM Serving

Emira Ibrahimović
emira@mit.edu

Medha Mahanta
medham@mit.edu

November 1, 2026

1 Introduction

Through our project we will test whether path bandwidth exceeding KV cache production rate is sufficient for schemes which simultaneously choose the prefill and decode nodes to achieve significant latency gains over schemes which delay the choice of decode node.

LLM inference systems often disaggregate prefill and decode [ZLC+24, PCZ+24], necessitating the transport of large KV caches between prefill and decode nodes, often across scale-ups. Early binding schedulers (e.g. Mooncake [QLH+25]) choose a prefill-decode node pair at request time, enabling prefill nodes to stream the KV cache calculated after each layer to the decode node, hiding the communication overhead behind prefill. However, the destination choice is fixed at request-time, so network conditions can affect the latency of cache transfers. Late binding methods (NetKV [**?**]) defer the choice of decode node until prefill finishes, adapting to current congestion and load. The cost is that the transport of the entire KV cache remains in the TTFT path. We expect early binding to achieve gains over late binding only while the path drains KV bytes at least as fast as prefill produces them. To identify the point when this happens, we will build a simulator based on LLMServingSim 2.0 (KAIST) [**?**] and Frontier (NetX-lab) [**?**] to model a two-tier leaf–spine with racks. By varying the oversubscription, prompt length and request type, we examine how the KV throughput affects the choice of optimal binding policy, evaluated on mean and p99 first- and second-token latency.

2 System

In inference systems with small NVLink domains, a single scale-up domain cannot hold prefill and decode instances at the P:D ratio the workload needs, so most KV caches must travel over slower Ethernet or InfiniBand links. The router therefore has to decide when to bind a request to a decode instance. Binding early, when the request arrives, lets the KV cache stream to the decode instance layer by layer during prefill, but the choice is made with stale load and congestion information. Binding late, after prefill finishes, uses fresh information but cannot overlap the transfer with prefill.

2.1 Goals

Through this project, we will produce a map from network and workload configurations to the best binding time. Our second goal is to propose a new binding policy based on our findings.

2.2 Modules

- **Target system.** We analyze a serving system with 128 NVIDIA H100 GPUs arranged into 16 NVLink domains of 8 GPUs each. We consider inference on Llama-3.1-70B, which uses GQA with 8 KV heads. With tensor parallelism of degree 4, each server hosts 2 instances. We sweep over the prefill:decode ratio. The servers are grouped into racks and connected by a two-tier leaf-spine network.

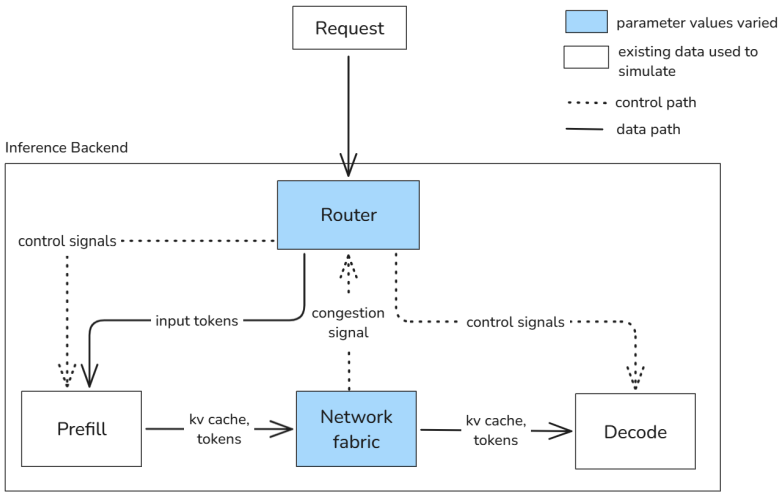


Figure 1: The disaggregated LLM serving system. The router binds each request to a decode node either on arrival or after prefill completes. The first approach enables layer-wise KV streaming during prefill, while the second one uses more relevant congestion information. Blue components will be modeled explicitly in our flow-level simulator; white components are calibrated from published profiling data.

- **Simulator.** The simulator explicitly models layer-wise KV streaming, background traffic on the network, and the oversubscription ratio (Fig. 1, "Network fabric"), which together set the effective bandwidth available to KV transfers. The router sees network congestion with a configurable delay. Prefill and decode latencies come from published profiling data [FTD+26, AKM+24]. Requests come from publicly available traces [PCZ+24, Mic23, QLH+25, Moo25].
- **Policies.** We compare binding policies in the router (Fig. 1, "Router"): early binding with layer-wise streaming, as in Mooncake [QLH+25]; late, network-aware binding, as in NetKV [Oje26]; and a hybrid that chooses between them per request. Time permitting, we will also design a policy that binds early and later moves the request to a better decode instance, using live KV migration as in Llumnix [SHZ+24], and see how it compares.

3 Implementation Plan

Table 1: Implementation plan.

Week	Milestone	Tasks
1 (Oct 19)	Model done, traces loading	Work out the latency model for early vs. late binding on paper. Figure out ρ for the papers we compare against. Get the Mooncake and Azure traces loading and collect prefill/decode timing numbers.
2 (Oct 26)	Basic simulator runs	Write the network part (leaf-spine topology, links shared fairly between flows) and test it. Write the request side: arrivals, prefill and decode queues, KV cache sizes.
3 (Nov 2)	All policies implemented	Add layer-by-layer KV streaming, background traffic and delayed congestion info. Implement early, late and threshold binding.
4 (Nov 9)	Simulator verified	Make sure we can reproduce results from DistServe, NetKV and ObjectCache. Compare simulator output to our model. Run a few setups in LLMServingSim and compare.

5 (Nov 16)	Main results	Run the main experiments, varying ρ, how bursty background traffic is, load, oversubscription and workload. Try new routing algorithms if we have time.
6 (Nov 23)	Results done	Fit the threshold rule, check how much results depend on our assumptions, make final plots. Thanksgiving.
7 (Nov 30)	Paper and presentation prep	Write the paper and make the slides.

References

[AKM+24] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[FTD+26] Yicheng Feng, Xin Tan, Yangtao Deng, Yimin Jiang, Yibo Zhu, and Hong Xu. Frontier: Towards comprehensive and accurate LLM inference simulation. https://github.com/NetX-lab/Frontier, 2026. arXiv:2605.21312.

[Mic23] Microsoft Azure. Azure LLM inference trace 2023. https://github.com/Azure/AzurePublicDataset, 2023. Code and conversation traces, released with Splitwise.

[Moo25] Moonshot AI. Mooncake request traces. https://github.com/kvcache-ai/Mooncake, 2025. Anonymized request traces released with the FAST 2025 paper, in the `FAST25-release` directory.

[Oje26] Mubarak Adetunji Ojewale. NetKV: Network-aware decode instance selection for disaggregated LLM inference, 2026. arXiv:2606.03910.

[PCZ+24] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.

[QLH+25] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, 2025. USENIX Association.

[SHZ+24] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 173–192. USENIX Association, 2024.

[ZLC+24] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, 2024.